

ASSIGNMENT-10.1

Name:U.Sushmitha

Batch:24

Ht.No:2303A51705

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

Calculate average score of a student

```
def calc_average(marks):
```

```
    total = 0
```

```
    for m in marks:
```

```
        total += m
```

```
    average = total / len(marks)
```

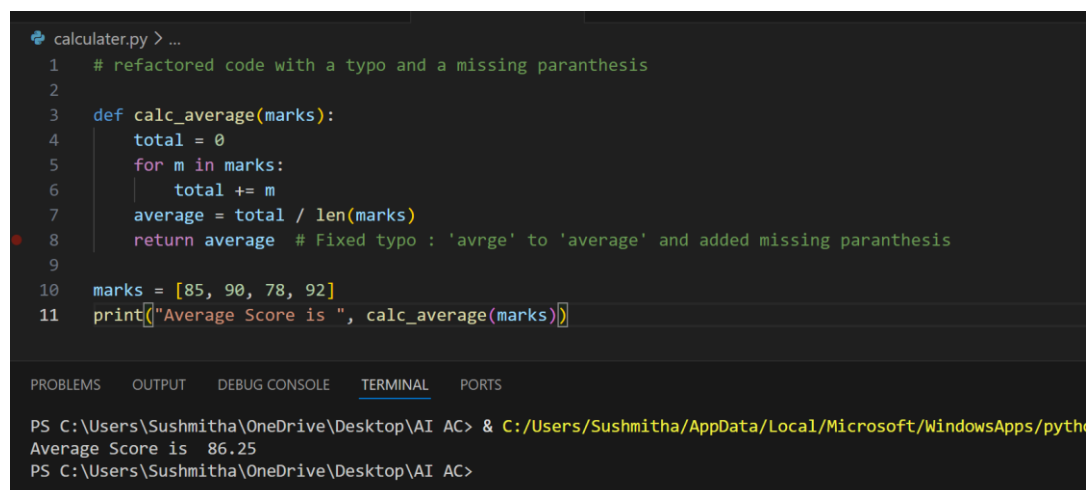
```
    return avrage # Typo here
```

```
marks = [85, 90, 78, 92]
```

```
print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.



```
calculator.py > ...
1  # refactored code with a typo and a missing paranthesis
2
3  def calc_average(marks):
4      total = 0
5      for m in marks:
6          total += m
7      average = total / len(marks)
8      return average # Fixed typo : 'avrge' to 'average' and added missing paranthesis
9
10 marks = [85, 90, 78, 92]
11 print("Average Score is ", calc_average(marks))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python
Average Score is 86.25
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #2 – PEP 8 Compliance

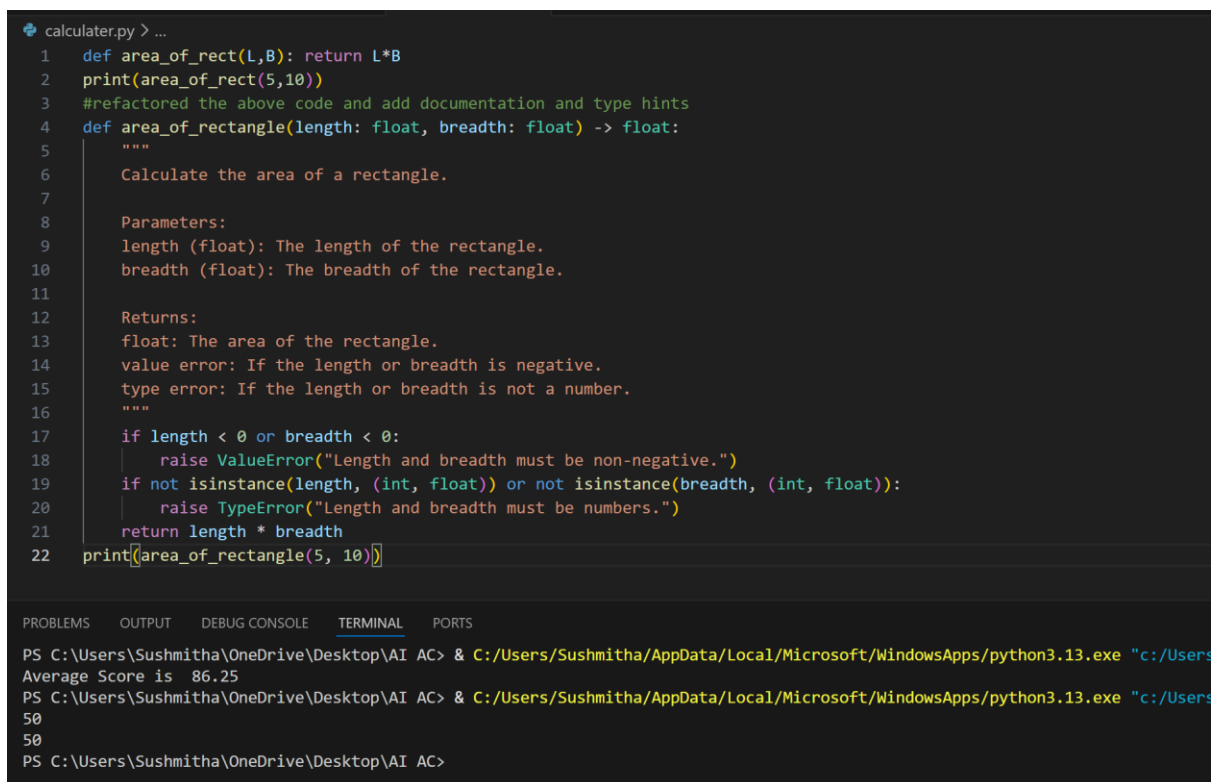
Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B  
  
print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.



```
calculator.py > ...  
1  def area_of_rect(L,B): return L*B  
2  print(area_of_rect(5,10))  
3  #refactored the above code and add documentation and type hints  
4  def area_of_rectangle(length: float, breadth: float) -> float:  
5      """  
6          Calculate the area of a rectangle.  
7  
8          Parameters:  
9          length (float): The length of the rectangle.  
10         breadth (float): The breadth of the rectangle.  
11  
12         Returns:  
13         float: The area of the rectangle.  
14         value error: If the length or breadth is negative.  
15         type error: If the length or breadth is not a number.  
16         """  
17         if length < 0 or breadth < 0:  
18             raise ValueError("Length and breadth must be non-negative.")  
19         if not isinstance(length, (int, float)) or not isinstance(breadth, (int, float)):  
20             raise TypeError("Length and breadth must be numbers.")  
21         return length * breadth  
22 print(area_of_rectangle(5, 10))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users  
Average Score is 86.25  
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users  
50  
50  
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

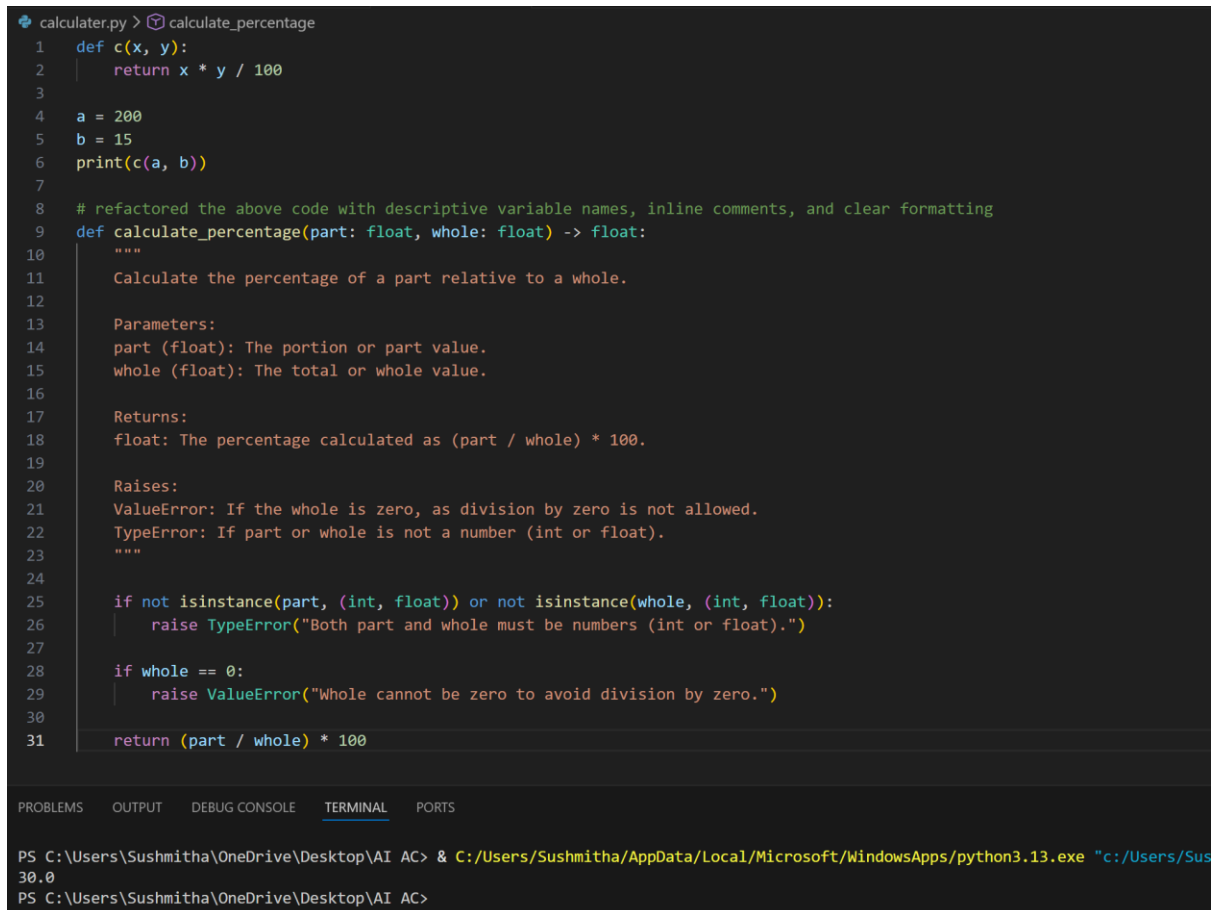
```
def c(x,y):  
  
    return x*y/100  
  
a=200
```

b=15

print(c(a,b))

Expected Output:

- Python code with descriptive variable names, inline comments, and clear formatting.



The screenshot shows a code editor with a file named 'calculator.py'. The editor displays two versions of a Python function. The first version (lines 1-6) is a simple function 'c(x, y)' that returns 'x * y / 100'. It is called with 'a = 200' and 'b = 15'. The second version (lines 8-31) is a refactored function 'calculate_percentage(part: float, whole: float) -> float:'. It includes a docstring with a description, parameters, returns, and raises sections. The function includes type hints, docstrings, and error handling for non-numeric inputs and division by zero. The terminal at the bottom shows the command to run the file using Python 3.13.

```
calculator.py > calculate_percentage
1  def c(x, y):
2      return x * y / 100
3
4  a = 200
5  b = 15
6  print(c(a, b))
7
8  # refactored the above code with descriptive variable names, inline comments, and clear formatting
9  def calculate_percentage(part: float, whole: float) -> float:
10     """
11     Calculate the percentage of a part relative to a whole.
12
13     Parameters:
14     part (float): The portion or part value.
15     whole (float): The total or whole value.
16
17     Returns:
18     float: The percentage calculated as (part / whole) * 100.
19
20     Raises:
21     ValueError: If the whole is zero, as division by zero is not allowed.
22     TypeError: If part or whole is not a number (int or float).
23     """
24
25     if not isinstance(part, (int, float)) or not isinstance(whole, (int, float)):
26         raise TypeError("Both part and whole must be numbers (int or float).")
27
28     if whole == 0:
29         raise ValueError("Whole cannot be zero to avoid division by zero.")
30
31     return (part / whole) * 100

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/Sus
30.0
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
```

```
print("Welcome", students[0])
```

```
print("Welcome", students[1])
```

```
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

```
calculator.py > welcome_student
1  students = ["Alice", "Bob", "Charlie"]
2  print("Welcome", students[0])
3  print("Welcome", students[1])
4  print("Welcome", students[2])
5
6  # refactored code to reduce redundancy with reusable function
7  def welcome_student(student: str) -> None:
8      """
9      Print a welcome message for a student.
10
11      Parameters:
12      student (str): The name of the student to welcome.
13
14      Returns:
15      None
16
17      values:
18      student: A string representing the name of the student.
19      type error: If the input is not a string, a TypeError will be raised.
20      """
21
22      if not isinstance(student, str):
23          raise TypeError("Student name must be a string.")
24
25      print("Welcome", student)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.1
Welcome Alice
Welcome Bob
Welcome Charlie
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

```
nums = [i for i in range(1,1000000)]
```

```
squares = []
```

```
for n in nums:
```

```
    squares.append(n**2)
```

```
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized

operations.

```
def is_valid_username(username): Untitled-1 • calculator.py X
calculator.py > ...
1  nums = [i for i in range(1, 1000000)]
2  squares = []
3  for n in nums:
4      squares.append(n**2)
5  print(len(squares))
6
7  # refactored the above code to reduce time complexity
8  nums = [i for i in range(1, 1000000)]
9  squares = [n**2 for n in nums]
10 print(len(squares))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsApps/python3.13.6
999999
999999
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

```
calculator.py > ...
1  import time
2
3  time1 = time.time()
4  nums = [i for i in range(1, 1000000)]
5  squares = []
6  for n in nums:
7      squares.append(n**2)
8  #print(len(squares))
9  time2 = time.time()
10 print("Time taken: ", time2 - time1)
11
12 # refactor the above code to reduce time complexity
13 time3 = time.time()
14 nums = [i for i in range(1, 1000000)]
15 squares = [n**2 for n in nums]
16 #print(len(squares))
17 time4 = time.time()
18 print("Time taken:", time4 - time3)
19
20 time5 = time.time()
21 #print(len([n**2 for n in range(1, 1000000)]))
22 time6 = time.time()
23 print("Time taken:", time6 - time5)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Users/Sushmitha/AppData/Local/Microsoft/WindowsAp
Time taken: 0.15569233894348145
Time taken: 0.12671947479248047
Time taken: 1.430511474609375e-06
PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):  
    if score >= 90:  
        return "A"  
    else:  
        if score >= 80:  
            return "B"  
        else:  
            if score >= 70:  
                return "C"  
            else:  
                if score >= 60:  
                    return "D"  
                else:  
                    return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

```

def is_valid_username(username):
    """
    Return the grade based on the score.

    Parameters:
    score (int): Student score

    Returns:
    str: Grade (A, B, C, D, or F)
    """
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"

print(grade(95))

def grade(score: int) -> str:
    """
    Return the grade based on the score using dictionary mapping.
    """
    grade_map = {
        90: "A",
        80: "B",
        70: "C",
        60: "D",
        0: "F"
    }

    for cutoff, letter in grade_map.items():
        if score >= cutoff:
            return letter

print(grade(85))

```

Time taken: 1.430511474609375e-06

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC> & C:/Us

A

B

PS C:\Users\Sushmitha\OneDrive\Desktop\AI AC>